Extra Methods of `Collectors`

- **`partitioningBy(Predicate)`**
  - Returns a `Collector` object that separates stream elements to two groups (in a map) with a given criterion.

```
Map<Boolean, List<Car>> carsPartitionedByPrice =
      cars.stream()
          .collect(partitioningBy(
                    (Car car)-> car.getPrice()>5000));
```

Keys	Values (List<T> by default)
True	List of cars that are > 5000
False	List of cars that are NOT > 5000

- **`partitioningBy(Predicate, Collector)`**
  - Returns a `Collector` object that separates stream elements to two groups (in a map) with a given criterion.
  - Can use a non-List collection for map values by supplying a `Collector`.

```
Map<Boolean, Map<Integer,Car>> carsByPrice =
      cars.stream()
          .collect(partioningBy(
                    (Car car)-> car.getPrice()>5000,
                   toMap((Car car)-> car.getId(),
                         (Car car)-> car)));
```

Keys	Values (Map<Integer, Car> in this example)
True	Map of cars that are > 5000
False	Map of cars that are NOT > 5000


```
Map<Boolean, Long> avgPriceByMakers =
      cars.stream().collect(partioningBy(
         (Car car)-> car.getPrice()>5000,
         counting());
```

Keys	Values (Double in this example)
True	Average car price, > 5000
False	Average car price, NOT > 5000

```
Map<Boolean, Double> avgPriceByMakers =
      cars.stream().collect(partioningBy(
         (Car car)-> car.getPrice()>5000,
         averagingDouble( car -> car.getPrice() ));
```

Keys	Values (Double in this example)
True	Average car price, > 5000
False	Average car price, NOT > 5000

- Other useful methods:
  - **`Collectors.maxBy(Comparator)`**
  - **`Collectors.minBy(Comparator)`**
  - **`Collectors.summarizing…(...)`**
    - Returns a **`SummaryStatistics`**
  - **`Collectors.mappping(Function, Collector)`**

```
- Map<Boolean, List> avgPriceByMakers =
        cars.stream().collect(partioningBy(
           (Car car)-> car.getPrice()>5000,
           mapping( car -> car.getPrice()
                    toList()));
```

Keys	Values (List in this example)
True	List of prices, > 5000
False	List of prices, NOT > 5000

- Use `collectingAndThen()` to do something on the partitioned result.

```
List<Long> counts =
   cars.stream().collect(collectingAndThen(
      partioningBy(car -> car.getPrice()>5000,
                   counting()),
      map -> List.of(map.get(true), map.get(false)) ));
```


- **`groupingBy(Function)`**
  - Returns a `Collector` object that groups stream elements (in a map) with a given criterion.

```
Map<String, List<Car>> carsGroupedByMakers =
      cars.stream().collect(
         groupingBy((Car car)-> car.getMaker());
```

Keys	Values (List<T> by default)
Ford	List of cars that are made by Ford
GM	List of cars that are made by GM
Honda	List of cars that are made by Honda

- **`groupingBy(Function, Collector)`**
  - Returns a `Collector` object that groups stream elements (in a map) with a given criterion.
  - Can use a non-List collection for map values by supplying a `Collector`.

```
Map<String, Double> avgPriceByMakers =
      cars.stream().collect(groupingBy(
         car -> car.getMaker(),
         averagingDouble( car -> car.getPrice() ));
```

Keys	Values (Double in this example)
Ford	7500.50
GM	6250.75
Honda	5175.25

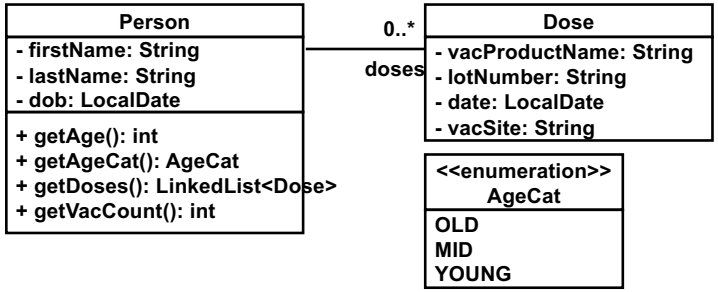
- **`teeing(Collector, Collector, BiFunction)`**
- Returns a `Collector` that runs two (downstream) `Collector`s and merges their results

```
double average =
   cars.stream().collect(teeing(
      summingInt( car -> car.getPrice() ),
      counting(),
      (sum, count) -> (double) sum/count ));
```

```
List<Integer> range =
   cars.stream().collect(teeing(
      minBy( Comparator.comparing(car->car.getPrice() ),
      maxBy( Comparator.comparing(car->car.getPrice() ),
      (min, max) -> List.of(min, max) ));
```

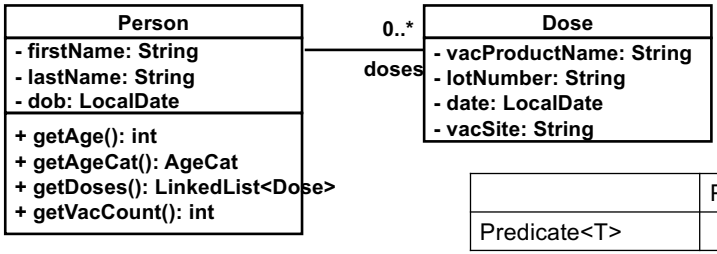
Exercise: Calculating Vaccination Rate

- Suppose you have a dataset about vaccination status in a certain community (e.g. town, college)
  - The dataset (in a DB or as a CSV file) has been parsed into a `LinkedList<Person>`.
  - Each `Person` has a `LinkedList<Dose>`.



- Compute the vaccination rate of this community (`LinkedList<Person>`).
  - Rate = # of fully vaccinated 18+ yr-olds / total headcount

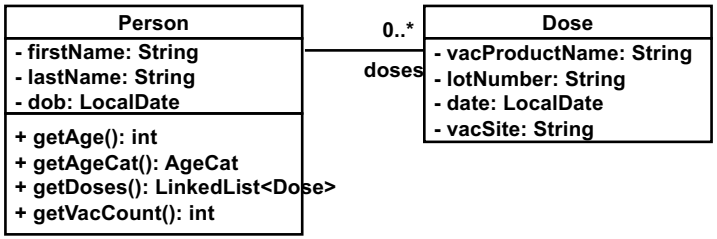
```
LinkedList<Person> people = ...;
long fullyVacCount = people.stream()
                           .filter(Person p)->
                                   {p.getAge()>=18 &&
                                    p.getVacCount()>=3})
                           .count();
float vacRate = (float)fullyVacCount/people.size() * 100;
```



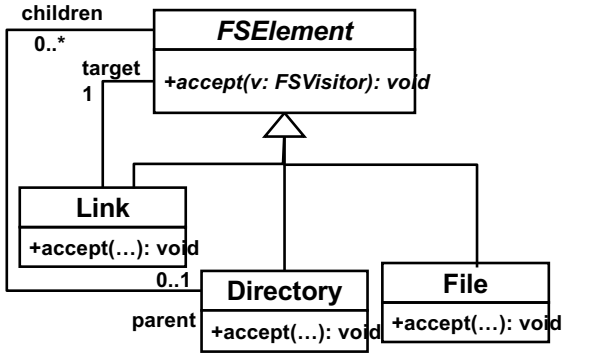
	Params	Returns
Predicate<T>	T	boolean

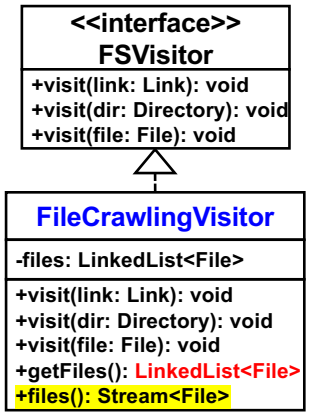
- Let's do the same with `collect()`.

```
LinkedList<Person> people = ...;
Map<Boolean, Double> vacCounts =
  people.stream()
        .collect(partitioningBy(
                 p->{p.getAge()>=18 && p.getVacCount()>=3},
                 averagingDouble(p-> p.getVacCount())));
```



Recap: HW 1






```
FileCrawlingVisitor visitor = new ...;
rootDir.accept( visitor );
visitor.files()
       ...;
```

HW 3

- Group files based on their file extensions and obtain a `SummaryStatistics` about file sizes for each group.
  - Use `groupingBy()`.
- Keep using a file system structure (test fixture) that you used for HW 1.

HW 4

- Process a list of `car`s with Stream API.
- Group `car`s based on their production years (i.e. in which year they were made) and obtain a `SummaryStatistics` about prices for each group.
  - Use `groupingBy()`

Reduce Operations in Map-Reduce Data Processing

- `Stream` provides…
  - "ready-made" methods for simple data processing tasks.
    - e.g. `count()`, `max()`, `min()`, `sum()`, `average()`
  - `collect()` to perform more complex data processing tasks.
  - `reduce()` to implement your own (custom) reduce operation..

Reduce Operations with `reduce()`

- Use `reduce()` if you implement your own (custom) reduce operation

```
- Optional<T>   reduce(BinaryOperator<T> accumulator)

- T             reduce(T initVal,
                       BinaryOperator<T> accumulator)

- U             reduce(U initVal,
                       BiFunction<U,T> accumulator,
                       BinaryOperator<U> combiner)
```

1st Version of `reduce()`

- `Optional<T> reduce(BinaryOperator<T> accumulator)`
  - Takes a reduction function (`accumulator`) as a LE.
    - Applies it on stream elements (`T`) one by one.
  - Returns the reduced value (`T`).

```
T result = aStream.reduce( (T result, T elem)-> {...} )
                  .get();
```

» `result`: result holder

» `elem`: next available element

	Params	Returns	Example use case
BinaryOperator<T>	T, T	T	Multiplying two numbers (*)

1st Version of `reduce()`

- `Optional<T> reduce(BinaryOperator<T> accumulator)`
  - Takes a reduction function (`accumulator`) as a LE.
    - Applies it on stream elements (`T`) one by one.
  - Returns the reduced value (`T`).

```
T result = aStream.reduce( (T result, T elem)-> {...} )
                  .get();
```

```
Iterator<T> it = collection.iterator();
T result = it.next();          // first element
while(it.hasNext()){           // for each remaining element
    T elem = it.next();
    result = accumulate(result, elem);
}
```

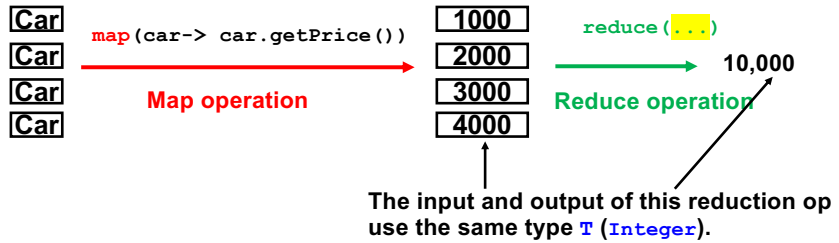

- `T result = aStream.reduce( (T result, T elem)-> {...} ).get();`

- ```
  Iterator<T> it = collection.iterator();
  T result = it.next();          // first element
  while(it.hasNext()){           // for each remaining element
      T elem = it.next();
      result = accumulate(result, elem); }
  ```

- `result` : *result holder*
  - is *initialized* with the first element.
  - is *updated* in each iteration of the loop by
    - Getting accumulated with the next element (`elem`) with `accumulate()`

- A reduce operation (`accumulator`) is implemented as an anonymous version of `accumulate()`.
  - `accumulator`'s code block == `accumulate()`'s method body


- ```
  Integer totalValue
    = cars.stream()
          .map( (Car car)-> car.getPrice() )
          .reduce((Integer result, Integer price)->{result+price})
          .get();
  ```



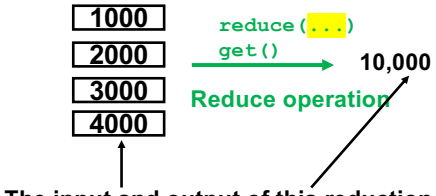
result = 1000
result = result + 2000 // 3,000
result = result + 3000 // 6,000
result = result + 4000 // 10,000

((1000 + 2000) + 3000) + 4000) = 10000

Important Notes (1)

```
LinkedList<Car> cars = ...;
Integer totalValue
  = cars.stream()
        .map( (Car car)-> car.getPrice() )
        .reduce((Integer result, Integer price)->{result+price})
        .get();
```

```
LinkedList<Car> cars = ...;
Integer totalValue
  = cars.stream()
        .map( (car)-> car.getPrice() )
        .reduce((result, price)->{result+price})
        .get();
```

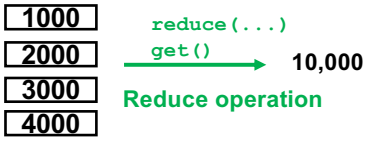


The input and output of this reduction op use the same type `T` (`Integer`).

Important Notes (2)

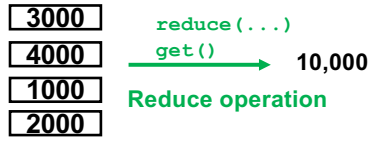
- The order of applying a LE on stream elements is **NOT** guaranteed.
  - Even though stream elements are ordered.
    - A stream's elements are ordered if its source collection is ordered (e.g., `List`).
- A reduction function must be **associative**.

```
Integer totalValue
    = cars.stream().map( (Car car)-> car.getPrice() )
                   .reduce( (result, price)->{result+price}).get();
```



result = 1000
result = result + 2000 // 3,000
result = result + 3000 // 6,000
result = result + 4000 // 10,000

((1000 + 2000) + 3000) + 4000) = 10000



result = 3000
result = result + 4000 // 7000
result = result + 1000 // 8000
result = result + 2000 // 10000

((3000 + 4000) + 1000) + 2000) = 10000

- Associative operator
  - (x op y) op z = x op (y op z)
  - Gives the same result regardless of the way the operands are grouped.
  - e.g., Numerical sum, numerical product, string concatenation, max, min, matrix product, set union, set intersection, logical AND, logical OR, logical XOR, etc.
- Non-associative operators
  - e.g., Numerical subtraction, numerical division, power, logical NOR, logical NAND, etc.
    - (10 - 5) - 2 = 3 V.S. 10 - (5 - 2) = 7
    - (10/5)/2 = 1 V.S. 10/(5/2) = 4
    - (10^5)^2 V.S. 10^(5^2)

2nd Version of `reduce()`

- `T reduce(T initVal, BinaryOperator<T> accumulator)`
  - Takes the initial value (`T`) for the reduced value (i.e. reduction result) as the first parameter.
  - Takes a reduction function (`accumulator`) as the second parameter.
    - Applies the function on stream elements (`T`) one by one.
  - Returns the reduced value (`T`).

```
T result = aStream.reduce(T initValue, (T result, T elem)-> {...});
```

» **`result`: result holder**

» **`elem`: next available element**

	Params	Returns	Example use case
BinaryOperator<T>	T, T	T	Multiplying two numbers (*)

- `T reduce(T initVal, BinaryOperator<T> accumulator)`
  - Takes the initial value (`T`) for the reduced value (i.e. reduction result) as the first parameter.
  - Takes a reduction function (`accumulator`) as the second parameter.
    - Applies the function on stream elements (`T`) one by one.
  - Returns the reduced value (`T`).

```
- T result = aStream.reduce(T initValue, (T result, T elem)-> {...});

- T result = initValue;
  for(T element: collection){
      result = accumulate(result, element);
  }
```

	Params	Returns	Example use case
BinaryOperator<T>	T, T	T	Multiplying two numbers (*)

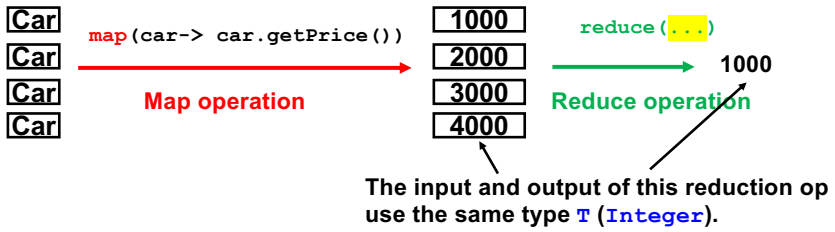

```
- T result = aStream.reduce(T initValue, (T result, T elem)-> {...});

- T result = initValue;
  for(T element: collection){
      result = accumulate(result, element);
  }
```

- `result` : *result holder*
  - is *initialized* with `initValue`.
  - is *updated* in each iteration of the loop by
    - Getting accumulated with the next element (`elem`) with `accumulate()`

- A reduce operation (`accumulator`) is implemented as an anonymous version of `accumulate()`.
  - `accumulator`'s code block == `accumulate()`'s method body


```
- Integer minPrice
    = cars.stream().map( (Car car)-> car.getPrice() )
                   . reduce(0, (result, carPrice)->{
                           if(result==0) return carPrice;
                           else if(carPrice < result) return carPrice;
                           else return result;} );
```



result = 0
result = 1000
result = 1000 (1000 < 2000)
result = 1000 (1000 < 3000)
result = 1000 (1000 < 4000)

- With `reduce()` in the Stream API

```
- Integer price = cars.stream()
                      .map( (Car car)-> car.getPrice() )
                      .reduce(0, (result, carPrice)->{
                             if(result==0) return carPrice;
                             else if(carPrice < result) return carPrice;
                             else return result;} );
```

- In a traditional style

```
- List<Integer> carPrices = ...
  int result = 0;
  for(Integer carPrice: carPrices){
          if(result==0) result = carPrice;
          else if(carPrice < result) result = carPrice;
          else result = result;
  }
```

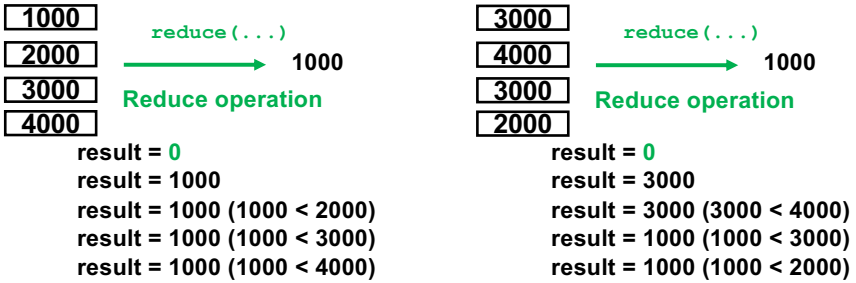
- With `min()` in the Stream API

```
- Integer price = cars.stream()
                      .map( (Car car)-> car.getPrice() )
                      .min( Comparator.comparing(price-> price) )
                      .get();
```

Important Note

- The order of applying a LE on stream elements is **NOT** guaranteed.
  - Even though stream elements are ordered.
    - A stream's elements are ordered if its source collection is ordered (e.g., `List`).

- A reduction operator must be **associative**.
  -
```
Integer price = cars.stream()
                    .map( (Car car)-> car.getPrice() )
                    .reduce(0, (result, carPrice)->{
                              if(result==0) return carPrice;
                              else if(carPrice < result) return carPrice;
                              else return result;} );
```

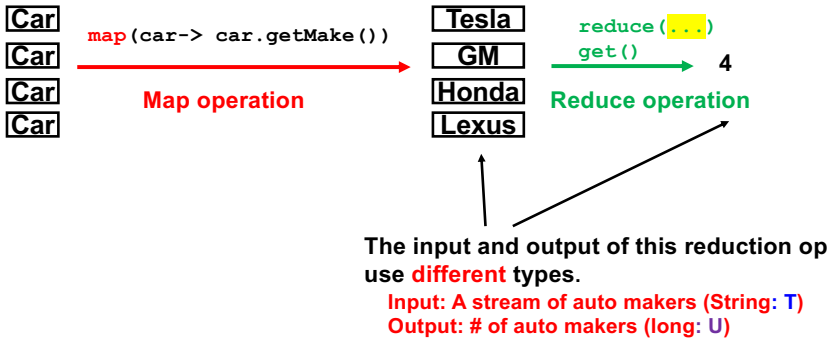


3rd Version of `reduce()`

- `U reduce(U initVal, BiFunction<U,T> accumulator, BinaryOperator<U> combiner)`
  - Takes the initial value (`U`) for the reduced value (i.e. reduction result) as the first parameter.
  - Takes a reduction function (`accumulator`) as the second parameter.
    - Applies the function on stream elements (`T`) one by one.
  - Takes a combination function (`combiner`) as the third parameter.
    - Applies the function on *intermediate* reduction results (`U`) one by one.
  - Returns the final (combined) result (`U`).
  - Useful when stream elements (`T`) and a reduced value (`U`) use different types.

	Params	Returns
BiFunction<U,T>	U, T	U
BinaryOperator<U>	U, U	U

- Implementing `count()` yourself with `reduce()`.
  -
```
long carMakerNum = cars.stream()
                       .map( (Car car)-> car.getMake() )
                       .count();
```



-
```
U finalResult = aStream.reduce(U initValue,
                               (U result, T element)-> {...},
                               (U finalResult, U intermediateResult)->...);
```
-
```
U result = initValue;
for(T element: collection){
    result = accumulate(result,element);
}
```

- `result` : *result holder*
  - is *initialized* with `initValue`.
  - is *updated* in each iteration of the loop by
    - Getting accumulated with the next element (`elem`) with `accumulate()`

- A reduce operation (`accumulator`) is implemented as an anonymous version of `accumulate()`.
  - `accumulator`'s code block == `accumulate()`'s method body

- With `reduce()` in the Streams API

```
long carMakerNum =
            cars.stream()
                .map( (Car car)-> car.getMake() )
                .reduce(0,
                        (result,carMaker)-> ++result,
                        (finalResult,intermediateResult)->finalResult);
```

- In traditional style

```
List<String> carMakers = ...
long result = 0;
for(String carMaker: carMakers){
    result++;
}
long carMakerNum = result;
```

- With `count()` in the Streams API

```
long carMakerNum = cars.stream()
                       .map( (Car car)-> car.getMake() )
                       .count();
```


Important Notes

- The order of applying a LE on stream elements is **NOT** guaranteed.
  - Even though stream elements are ordered.
    - A stream's elements are ordered if its source collection is ordered (e.g., `List`).
- Reduction and combination operators must be **associative**.

- With `reduce()` in the Stream API

```
long carMakerNum =
            cars.stream()
                .map( (Car car)-> car.getMake() )
                .reduce(0,
                  (result,carMaker)-> ++result,
                  (finalResult,intermidiateResult)->finalResult);
```

- `reduce()` executes `result = ++result;`
- Just in case, note that:

```
int i,x,y = 0; i++;          // i==1
int i,x,y = 0; int x = i++; // i==1, x==0
                             // assignment of 0 first
int i,x,y = 0; int y = ++i; // i==1, y==1
                             // increment of 0 first
```


- Just return `finalResult` (the first parameter) in the second LE unless you use a parallel stream in a multi-threaded app.

```
long carMakerNum =
            cars.stream()
                .map( (Car car)-> car.getMake() )
                .reduce(0,
                        (result,carMaker)-> ++result,
                        (finalResult,intermidiateResult)->finalResult);
```

- When using a default (i.e. non-parallel, sequential) stream with a single thread, you never need to do anything extra in the second LE.
  - Just return what's at hand now, which is contained in the first parameter of the second LE, as the final map-reduce result.

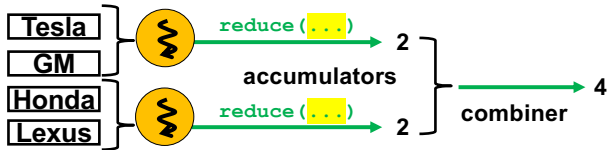
```
Collection<Long> intermediateResults = ...
Iterator<Long> it = intermediateResults.iterator();
Long finalResult = it.next();       // first element

while(it.hasNext()){                // for each remaining element
    Long intermediateResult = it.next();
    finalResult = combine(finalResult, intermediateResult);}
```

  - Using a sequential stream, there is ONLY ONE intermediate result.

- If you use a parallel stream, it
  - Uses multiple threads to run the first LE (reduction function) in a parallel/concurrent manner.
    - Each thread generates one intermediate reduction result.
  - combines all intermediate reduction results.

```
long carMakerNum =
  cars.stream()
      .parallel()
      .map( (Car car)-> car.getMake() )
      .reduce(
          0,
          (result,carMaker)-> ++result,
          (finalResult,intermidiateResult)->finalResult+intermediateResult);
```



3 Versions of reduce()

- If the input (stream elements) and output (reduced result) use the same type, use the 1st or 2nd version:
  - `Optional<T> reduce(BinaryOperator<T> accumulator)`
  - `T reduce(T initVal, BinaryOperator<T> accumulator)`
    - Use the 2nd version if you need a custom initial value.
- If the input (stream elements) and output (reduced result) use different types, use the 3rd version:
  - `U reduce(U initVal, BiFunction<U,T> accumulator, BinaryOperator<U> combiner)`